

Informe de Desarrollo

Práctica 1

Lenguajes Formales y de Programación · 2S2026

Diego Alejandro Barrios Gomez

201900158

Indice

Indice	2
Introducción	3
Análisis inicial del problema	3
Decisiones de diseño	4
3.1 Un archivo por responsabilidad, sin subcarpetas	4
3.2 Listas con funciones de búsqueda	4
3.3 El intento guarda su propio resultado	4
3.4 Errores como texto, no como excepciones	4
4. Retos técnicos encontrados	4
4.1 Reto principal: recorrer las cajas de 3×3	4
4.2 Verificar una unidad sin recorrerla varias veces	5
4.3 Distinguir "tablero válido" de "intento correcto"	5
4.4 Una línea mala arruinaba todo el archivo	5
4.5 Integridad referencial entre archivos	6
4.6 Ordenar los datos de los reportes	6
4.7 Rutas de archivo dependientes del directorio de ejecución.....	6
4.8 Los porcentajes se mostraban con un solo decimal.....	6
Pruebas realizadas	7
5.1 Datos de prueba	7
5.2 Casos de validación	7
5.3 Pruebas del menú.....	7
5.4 Verificación de estilo.....	7
5.5 Entornos probados	7
Conclusiones.....	8

Introducción

Este informe describe el proceso de construcción del motor de calificación y análisis del torneo de Sudoku solicitado por Numerix Academy, los retos técnicos que surgieron principalmente en la validación matricial y las decisiones de diseño que se tomaron para resolverlos.

El objetivo del sistema no es solo registrar resultados, sino razonar sobre la estructura del tablero para determinar qué tan correcto es un intento aun cuando el jugador no lo haya completado perfectamente. Esa exigencia definió buena parte de la organización del código.

Análisis inicial del problema

Antes de escribir código se identificaron tres problemas distintos que a primera vista parecían uno solo:

1. **Problema de lectura.** Convertir texto plano separado por comas en objetos, verificando que cada campo tenga sentido.
2. **Problema de estructura.** Convertir una cadena lineal de 81 caracteres en una representación bidimensional que permita razonar sobre filas, columnas y cajas.
3. **Problema de relación.** Cruzar información entre tres archivos independientes para producir estadísticas agrupadas.

Separarlos desde el inicio fue lo que llevó a dividir el programa en cinco archivos en lugar de escribirlo todo en uno. Cada problema quedó aislado en su propio archivo.

Decisiones de diseño

3.1 Un archivo por responsabilidad, sin subcarpetas

Los cinco archivos .py están juntos en la carpeta principal. Se consideró agruparlos en subcarpetas, pero eso obliga a crear archivos __init__.py y a escribir importaciones más largas, lo que agrega complejidad sin beneficio para un proyecto de este tamaño.

La separación por responsabilidad sí se mantuvo: clases.py define los moldes, lectura.py lee archivos, validacion.py aplica las reglas del Sudoku, reportes.py calcula y escribe, y main.py coordina. Las dependencias apuntan en una sola dirección, sin importaciones circulares.

3.2 Listas con funciones de búsqueda

Los tableros y los jugadores se guardan en listas, y para encontrar uno concreto se escribieron funciones que recorren la lista comparando el identificador:

```
def buscar_tablero(lista_tableros, id_sudoku):
    for tablero in lista_tableros:
        if tablero.id_sudoku == id_sudoku:
            return tablero
    return None
```

Existe la alternativa de usar diccionarios, que encontrarían el elemento sin recorrer nada. Se optó por las listas porque el volumen de datos es pequeño (ocho tableros y diez jugadores) y porque el recorrido explícito deja visible la operación que se está haciendo. La función devuelve None cuando no encuentra nada, lo que permite detectar los intentos que apuntan a registros inexistentes.

3.3 El intento guarda su propio resultado

En lugar de devolver los resultados de la calificación en una estructura separada, la función calificar_intento() los escribe dentro del propio objeto Intento. Esto simplificó los reportes: cada uno recibe la lista de intentos ya calificados y lee los atributos directamente, sin tener que mantener sincronizadas dos colecciones paralelas.

3.4 Errores como texto, no como excepciones

Las líneas mal escritas no son fallas del programa: son datos de entrada imperfectos, un resultado esperado. Por eso cada función de carga devuelve dos listas —los objetos creados y los mensajes de error— en lugar de lanzar excepciones. El menú imprime ambas.

4. Retos técnicos encontrados

4.1 Reto principal: recorrer las cajas de 3×3

Las filas y las columnas son directas de extraer de una matriz: para la fila se fija el primer índice y para la columna el segundo. Las cajas de 3×3 fueron el punto más delicado, porque hay que traducir un número de caja (0 a 8) a las coordenadas donde empieza esa caja dentro de la matriz.

Primer enfoque descartado. La idea inicial fue recorrer las cajas con dos ciclos anidados sobre bloques, uno para la banda horizontal y otro para la vertical. Eso obligaba a llevar dos contadores separados y rompía la simetría con las filas y columnas, que sí se recorren con un solo índice de 0 a 8.

Solución adoptada. Calcular la esquina de cada caja a partir de un único número usando división entera y residuo:

```
fila_inicial = (numero_caja // 3) * 3
```

```
columna_inicial = (numero_caja % 3) * 3
```

La división entera indica en qué banda horizontal está la caja, y el residuo en qué banda vertical. Con esto las tres unidades se recorren con el mismo índice de 0 a 8, lo que permitió escribir la calificación como tres ciclos idénticos y dejó claro de dónde sale el denominador de 27.

4.2 Verificar una unidad sin recorrerla varias veces

Comprobar que una fila contenga los dígitos del 1 al 9 sin repetirse parecía requerir varias pasadas: una para buscar duplicados, otra para confirmar que no hubiera ceros y otra para verificar el rango.

Solución. Invertir el recorrido: en vez de revisar los valores buscando problemas, se revisan los dígitos del 1 al 9 contando cuántas veces aparece cada uno.

```
for numero_buscado in range(1, 10):
    veces_que_aparece = 0
    for valor in valores:
        if valor == numero_buscado:
            veces_que_aparece = veces_que_aparece + 1
    if veces_que_aparece != 1:
        return False
return True
```

Si algún dígito no aparece exactamente una vez, la unidad está mal. Eso cubre los tres casos de una sola vez: un repetido aparece 2 o más veces, un faltante aparece 0 veces, y si hay una celda vacía el 0 desplaza a algún dígito que entonces también aparece 0 veces.

4.3 Distinguir "tablero válido" de "intento correcto"

Un caso que no era obvio al leer el enunciado: un jugador puede entregar un tablero **perfectamente válido según las reglas del Sudoku** que sin embargo no corresponde al tablero que se le asignó, porque modificó las pistas iniciales.

Ese intento obtendría 100% de validez, pero no debería contar como resuelto.

Solución. Se trataron como dos condiciones independientes:

- `porcentaje_validez == 100` — el tablero cumple las reglas
- `pistas_respetadas == True` — el tablero es el que se le asignó

y el veredicto final exige ambas. Durante las pruebas aparecieron varios casos con 88.89% de validez descartados por haber alterado una sola pista, lo que confirmó que la distinción era necesaria.

4.4 Una línea mala arruinaba todo el archivo

En la primera versión, al encontrar un registro mal escrito la función terminaba de golpe y no cargaba nada. Un solo error dejaba al sistema sin datos.

Solución. Acumular los errores en una lista y usar `continue` para saltar a la siguiente línea. Las líneas correctas sí se cargan, y al final se muestra un resumen con el número de línea y el motivo de cada descarte:

Intentos cargados correctamente: 55

Registros descartados por formato incorrecto: 3

- Línea 56: la solución debe tener 81 dígitos y tiene 5
- Línea 57: el tiempo 'abc' no es un número entero
- Línea 58: la fecha '2026-03-16' no tiene el formato DD-MM-AAAA

4.5 Integridad referencial entre archivos

Un problema distinto al de formato: un intento puede estar perfectamente escrito pero apuntar a un carnet o a un id_sudoku que no existe en los otros archivos. Esto no se puede detectar al leer la línea, porque en ese momento los otros archivos podrían no estar cargados todavía.

Solución. Trasladar esta verificación al momento de la calificación (opción 4), cuando ya están las tres listas en memoria. Los intentos sin referencia se separan y se reportan aparte:

Intentos descartados porque el carnet o el sudoku no existen: 2

- Sudoku 3: no existe el carnet 202099999
- Carnet 202011234: no existe el sudoku 99

4.6 Ordenar los datos de los reportes

Los reportes necesitan mostrar la información ordenada: los jugadores de mejor a peor, los tiempos de menor a mayor, los tableros del más difícil al más fácil.

Solución. Se implementó el método de la burbuja en tres funciones separadas, una para cada criterio. El método compara cada elemento con el que le sigue y los intercambia si están al revés, repitiendo el recorrido hasta que todo queda ordenado.

Un detalle del funcionamiento: después de la primera vuelta el elemento mayor ya quedó al final, así que el ciclo interno recorre uno menos cada vez (cantidad - 1 - vuelta). Sin ese ajuste el algoritmo funcionaría igual pero haría comparaciones innecesarias.

Las funciones modifican la lista recibida directamente, sin devolver una nueva, porque en Python las listas se pasan por referencia.

4.7 Rutas de archivo dependientes del directorio de ejecución

Al ejecutar el programa desde una carpeta distinta a la del proyecto, las rutas hacia entrada/ fallaban.

Solución. Calcular las rutas a partir de la ubicación real del archivo:

```
RUTA_BASE = os.path.dirname(os.path.abspath(__file__))  
CARPETA_ENTRADA = os.path.join(RUTA_BASE, "entrada")
```

La variable `__file__` la crea Python automáticamente y contiene la ruta del archivo que se está ejecutando.

Con esto el programa encuentra sus archivos sin importar desde dónde se invoque, y `os.path.join` mantiene la compatibilidad entre Windows y Linux, que usan separadores distintos.

4.8 Los porcentajes se mostraban con un solo decimal

Al usar `round(numero, 2)` y convertir a texto, un valor como 50.0 quedaba como "50.0" en lugar de "50.00", y las columnas de los reportes se veían desalineadas.

Solución. Se escribió la función `formato_porcentaje()`, que separa la parte entera de la decimal y agrega los ceros que falten hasta completar dos posiciones.

Pruebas realizadas

5.1 Datos de prueba

Se construyó un conjunto de datos con soluciones verificadas: 8 tableros con dificultades crecientes (de 41 a 21 pistas), 10 jugadores distribuidos en los tres niveles y 55 intentos donde el desempeño varía según el nivel declarado.

Se incluyeron **deliberadamente** registros defectuosos para probar el manejo de errores:

Caso de prueba	Resultado esperado	Resultado obtenido
Carnet no numérico (20201XXXX)	Descartado con mensaje	Correcto
Nivel fuera del dominio (Avanzado)	Descartado con mensaje	Correcto
Solución de 5 caracteres	Descartado con mensaje	Correcto
Tiempo no numérico (abc)	Descartado con mensaje	Correcto
Fecha en formato ISO (2026-03-16)	Descartado con mensaje	Correcto
Intento con carnet inexistente	Descartado al calificar	Correcto
Intento con id_sudoku inexistente	Descartado al calificar	Correcto
Ruta de archivo inexistente	Mensaje de error, sin caída del programa	Correcto

5.2 Casos de validación

Caso	Validez esperada	Marcado como resuelto
Solución completamente correcta	100%	Sí
Solución correcta pero con una pista alterada	88.89%	No
Dos valores intercambiados en una fila	Parcial (fallan columnas y cajas)	No
Varios errores dispersos	Menor mientras más errores	No

Resultado global sobre los datos de prueba: **53 intentos calificados, 17 resueltos correctamente, 36 con al menos un error.**

5.3 Pruebas del menú

Acción	Comportamiento esperado	Resultado
Pedir un reporte sin calificar	Aviso y regreso al menú	Correcto
Calificar sin cargar archivos	Aviso con el conteo de cada lista	Correcto
Escribir una opción que no existe	Aviso y regreso al menú	Correcto
Escribir texto en lugar de número	Aviso y regreso al menú	Correcto
Cargar un archivo después de calificar	Se borra la calificación anterior	Correcto

5.4 Verificación de estilo

El código fue revisado con `pycodestyle` (límite de 100 columnas) sin advertencias, cumpliendo el estándar PEP 8 requerido.

5.5 Entornos probados

- Python 3.12 en Windows con PowerShell y Visual Studio Code
- Python 3.12 en Linux con terminal bash

Conclusiones

1. La elección de la representación de datos determinó la dificultad del problema. Convertir la cadena de 81 caracteres en una matriz 9×9 desde el constructor de las clases hizo que toda la validación posterior se redujera a recorrer índices. Trabajar directamente sobre la cadena habría obligado a calcular desplazamientos en cada verificación.
2. Unificar el recorrido de las tres unidades fue la simplificación más importante. Traducir el número de caja con $(\text{numero} // 3) * 3$ y $(\text{numero} \% 3) * 3$ permitió tratar filas, columnas y cajas con el mismo índice, y de ahí salió naturalmente el denominador de 27 que exige la fórmula del porcentaje.
3. Invertir la dirección de la búsqueda simplificó la verificación. Contar cuántas veces aparece cada dígito del 1 al 9, en lugar de revisar los valores buscando problemas, resolvió con un solo recorrido lo que parecía necesitar tres.
4. Separar la validación de formato de la validación de integridad referencial fue necesario. Son problemas de naturaleza distinta y ocurren en momentos distintos del flujo: uno al leer cada línea, el otro cuando ya están las tres listas en memoria.
5. Manejar los errores por registro hace al sistema utilizable en condiciones reales. Un archivo con una línea corrupta sigue siendo aprovechable, y el usuario recibe información precisa sobre qué corregir y dónde.
6. La separación por responsabilidad se pagó sola al final del desarrollo. Agregar dos reportes adicionales no requirió tocar las clases ni la validación, lo que confirma que las responsabilidades quedaron bien delimitadas.
7. Distinguir entre "tablero válido" e "intento correcto" reveló un caso que el enunciado insinuaba pero no explicitaba: la validez estructural del Sudoku y la fidelidad al tablero asignado son propiedades independientes, y solo cumpliendo ambas se considera resuelto un intento.